Лабораторна робота 7

Спостереження за TrackHub та відкат за метриками

Середовище: Docker Desktop, Minikube, Kubernetes, Argo CD, Argo Rollouts, Prometheus, Grafana, Loki та Alloy.

Дата підсумкової перевірки: 11 листопада 2026 року. Контрольний експеримент із помилковою версією проведено 5 листопада 2026 року.

Мета: налаштувати метрики й логи через GitOps, виміряти роботу застосунку під навантаженням та перевірити автоматичне припинення невдалої canary-ревізії.

Підсумок: компоненти моніторингу працюють, шість із шести цілей TrackHub доступні Prometheus, JSON-логи надходять у Loki. Помилкову ревізію зупинено, стабільна ревізія має 6/6 готових pod.

1 Розгортання моніторингу через GitOps

Завдання: розгорнути Prometheus, Alertmanager, Grafana, Loki та Alloy через Argo CD і перевірити їхній стан.

Виконання: конфігурацію Applications додано в argocd-apps. Команди kubectl get nodes, kubectl get applications -n argocd та kubectl get pods -n monitoring показали стан кластера.

Результат: вузол Ready, усі Applications Synced/Healthy, поди моніторингу Running.

```
PS C:\Users\Vlad\Documents\Навчання\DevOps> kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   11d   v1.37.0
PS C:\Users\Vlad\Documents\Навчання\DevOps> kubectl get applications -n argocd
NAME            SYNC STATUS   HEALTH STATUS
alloy           Synced        Healthy
argo-rollouts   Synced        Healthy
loki            Synced        Healthy
monitoring      Synced        Healthy
trackhub        Synced        Healthy
trackhub-root   Synced        Healthy
PS C:\Users\Vlad\Documents\Навчання\DevOps>
```

Рисунок 1. Вузол Kubernetes та стан Argo CD

```
PS C:\Users\Vlad\Documents\Навчання\DevOps> kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS        AGE
alertmanager-kube-prometheus-stack-alertmanager-0        2/2     Running   2 (4m58s ago)   6d
alloy-q4db8                                              2/2     Running   2 (4m58s ago)   6d
kube-prometheus-stack-grafana-7f48b678d9-snwgt           3/3     Running   0               3m35s
kube-prometheus-stack-kube-state-metrics-645c589fb6-vflcl   1/1     Running   2 (4m58s ago)   6d
kube-prometheus-stack-operator-7cdc8864dd-wkr6x          1/1     Running   2 (4m58s ago)   6d
kube-prometheus-stack-prometheus-node-exporter-kbb4t     1/1     Running   2 (4m58s ago)   6d
loki-0                                                   2/2     Running   4 (4m58s ago)   6d
loki-gateway-cd47c7dc8-zfwk4                             2/2     Running   2 (4m58s ago)   6d
prometheus-kube-prometheus-stack-prometheus-0            2/2     Running   2 (4m58s ago)   5d23h
PS C:\Users\Vlad\Documents\Навчання\DevOps>
```

Рисунок 2. Поди моніторингу

2 Метрики та ServiceMonitor

Завдання: експортувати лічильник HTTP-запитів і гістограму затримки, налаштувати ServiceMonitor та перевірити збір метрик.

Виконання: TrackHub віддає /metrics. Лічильник має мітки method, route і status. У Prometheus виконано запит sum(rate(trackhub_http_requests_total{namespace="trackhub"}[5m])).

Результат: шість із шести цілей UP; поточне значення запиту на скріні становить приблизно 1,80 запиту/с.

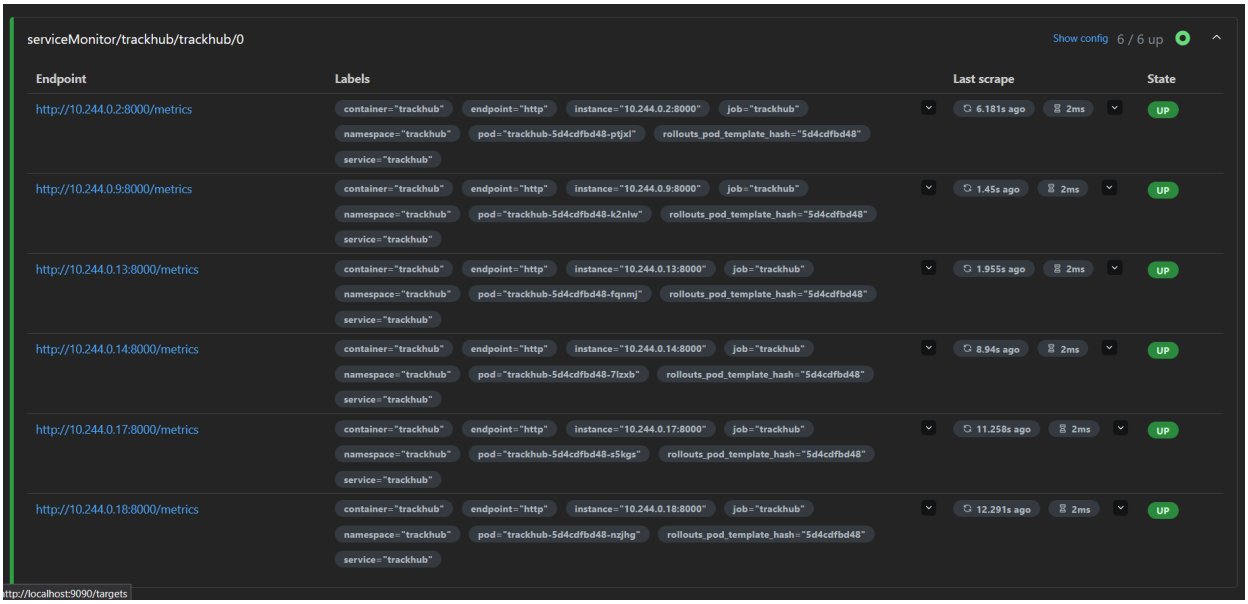


Рисунок 3. Цілі TrackHub: 6/6 UP

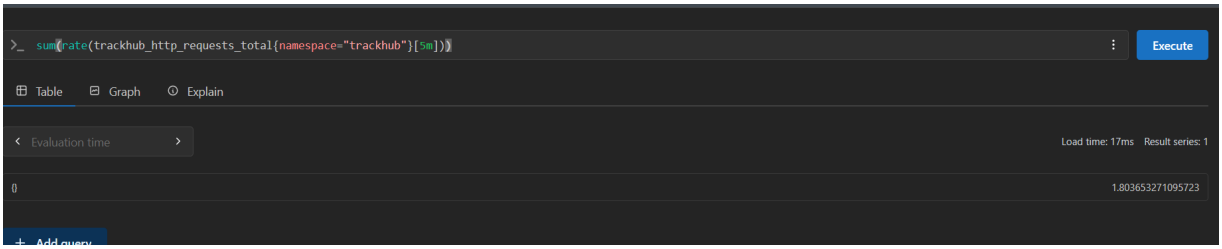


Рисунок 4. PromQL-запит і результат

3 Структуровані логи та Loki

Завдання: надсилати JSON-логи до Loki та знаходити HTTP-помилки через Grafana Explore.

Виконання: застосунок пише time, level, message, method, route, status і duration_ms у stdout. Alloy передає записи до Loki. Запит LogQL: {namespace="trackhub",container="trackhub"} | json. Для помилок використано status >= 500.

Результат: поточні логи доступні у Grafana; історичний скрін показує помилки canary-пода.

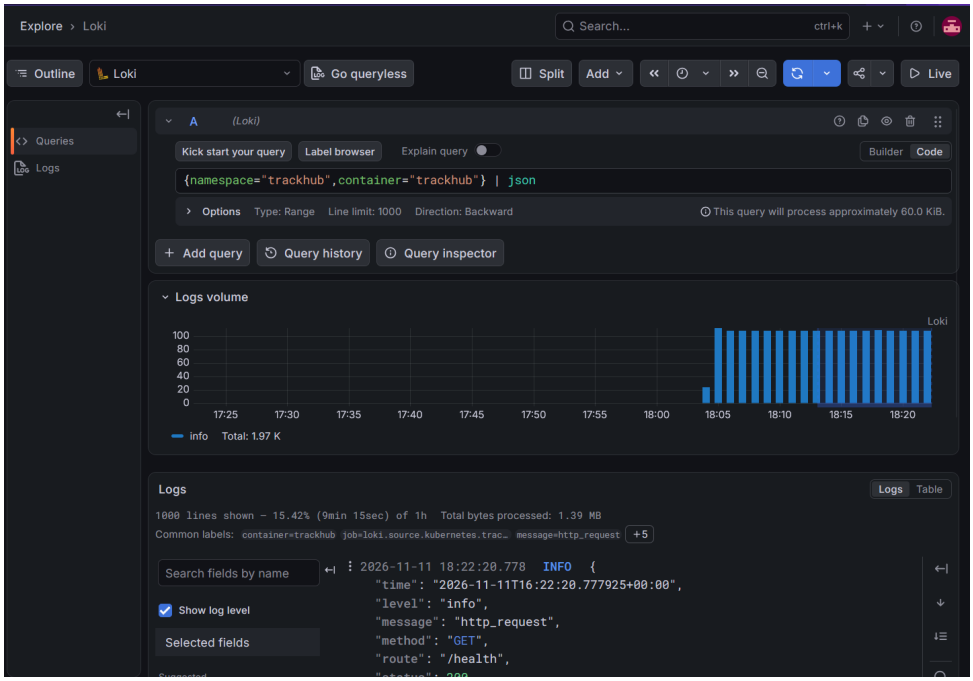


Рисунок 5. JSON-логи TrackHub

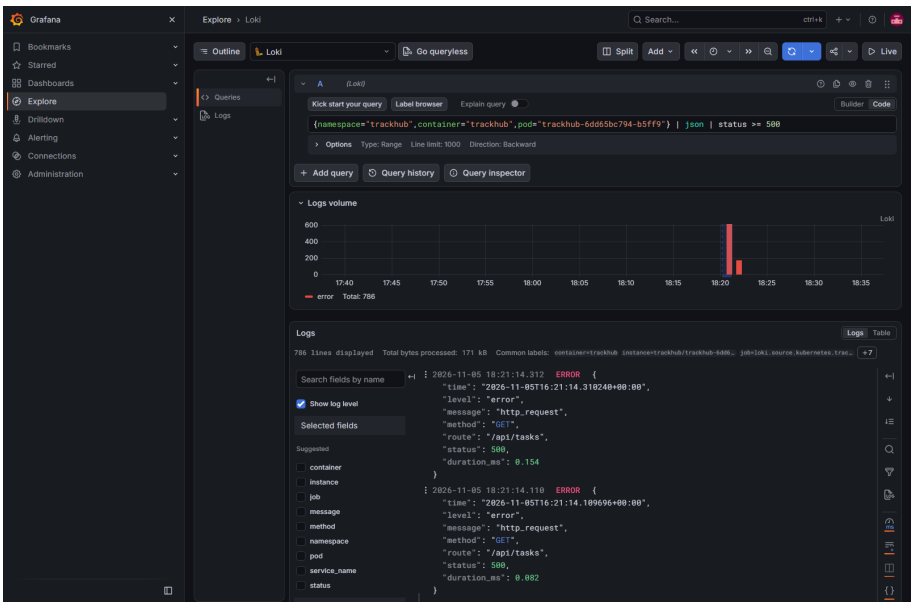


Рисунок 6. Фільтр HTTP 5xx під час контрольного тесту

4 Дашборд Grafana

Завдання: додати дашборд у Git і показати RPS, частку 5xx, p95, CPU, пам'ять і логи.

Виконання: JSON-документ збережено в charts/trackhub/dashboards; Helm доставляє його через ConfigMap із міткою grafana_dashboard.

Результат: усі шість панелей працюють. У здоровому стані частка 5xx дорівнює 0%.

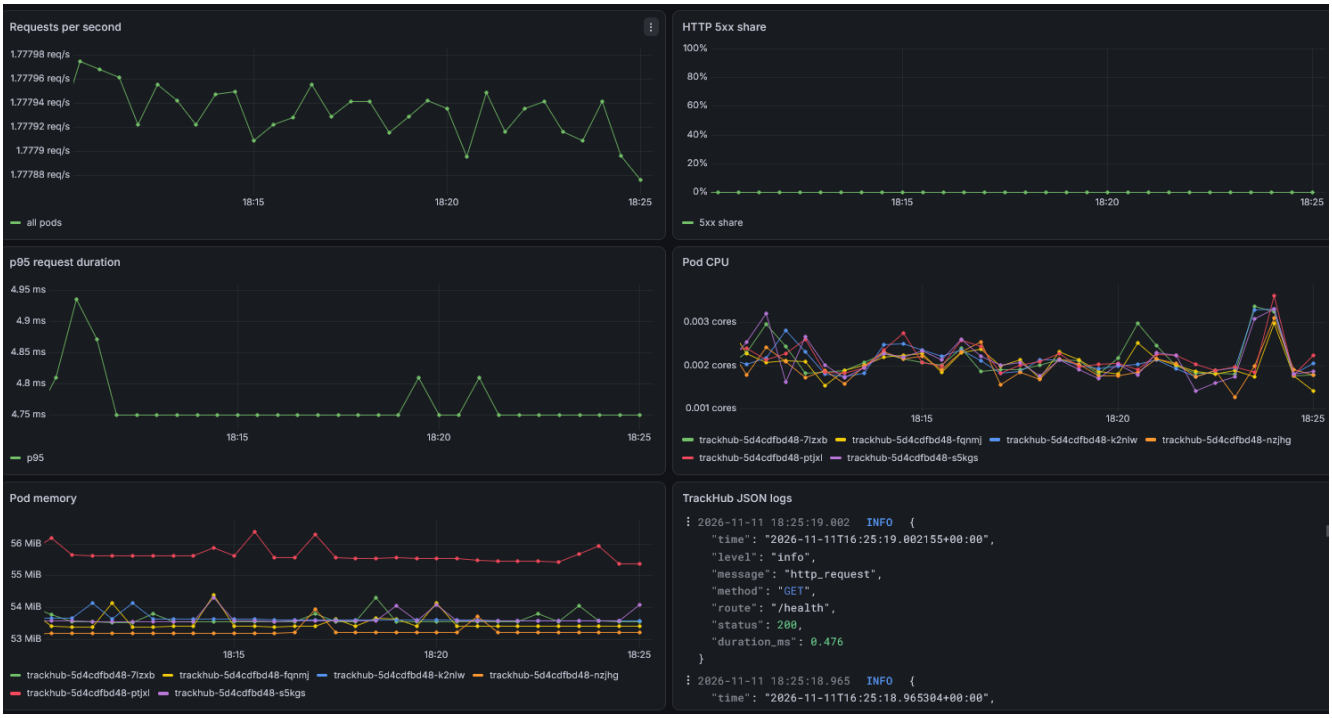


Рисунок 7. Дашборд TrackHub

5 Навантаження та масштабування

Завдання: запустити генератор запитів, спостерігати метрики та змінити кількість реплік через Git.

Виконання: scripts/lab7_observability_load.py працював 180 секунд із 8 потоками. Кількість реплік раніше змінено з 5 до 6 через Git і застосовано Argo CD. Повторне навантаження 11 листопада йшло через kubectl port-forward на один pod.

Результат: 7 640 запитів; 7 632 HTTP 200, 8 транспортних помилок, 0 HTTP 500. RPS, p95, CPU і пам'ять зросли. Цей повторний тест не вимірює рівномірність розподілу між усіма pod.

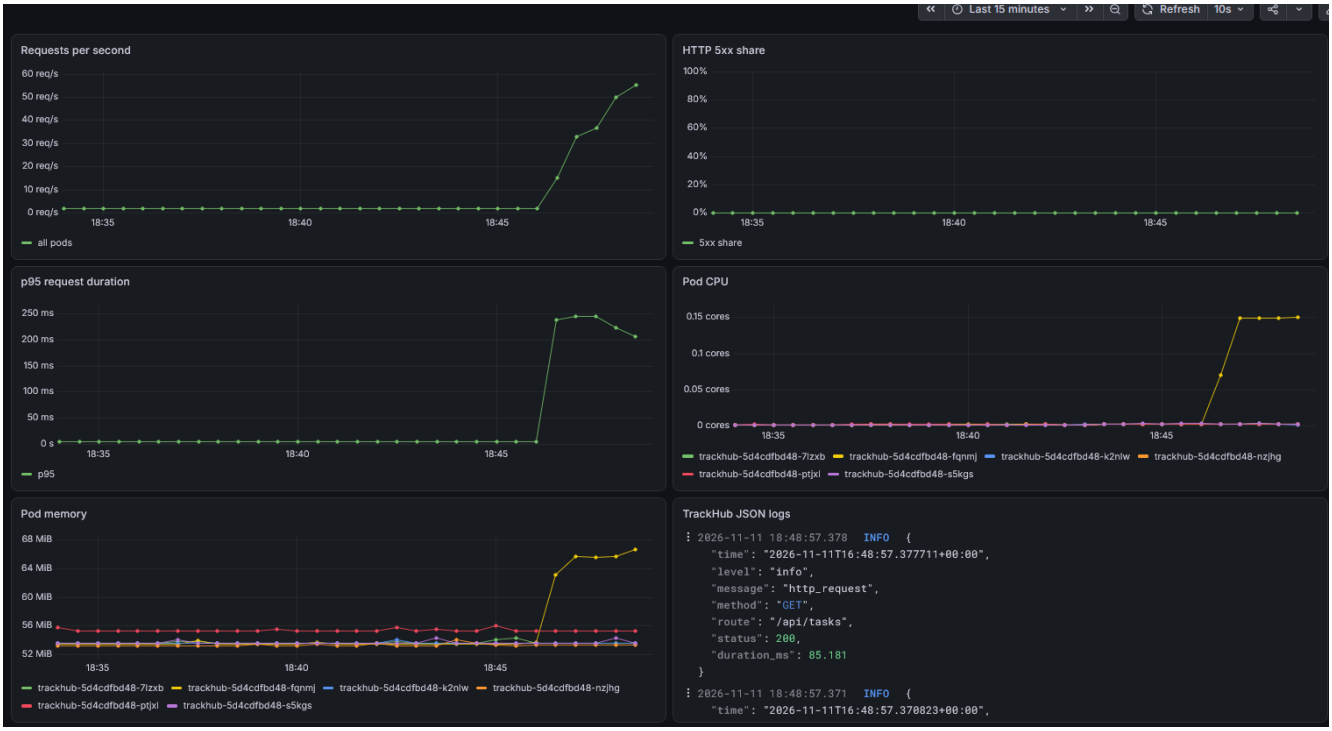


Рисунок 8. Метрики під навантаженням

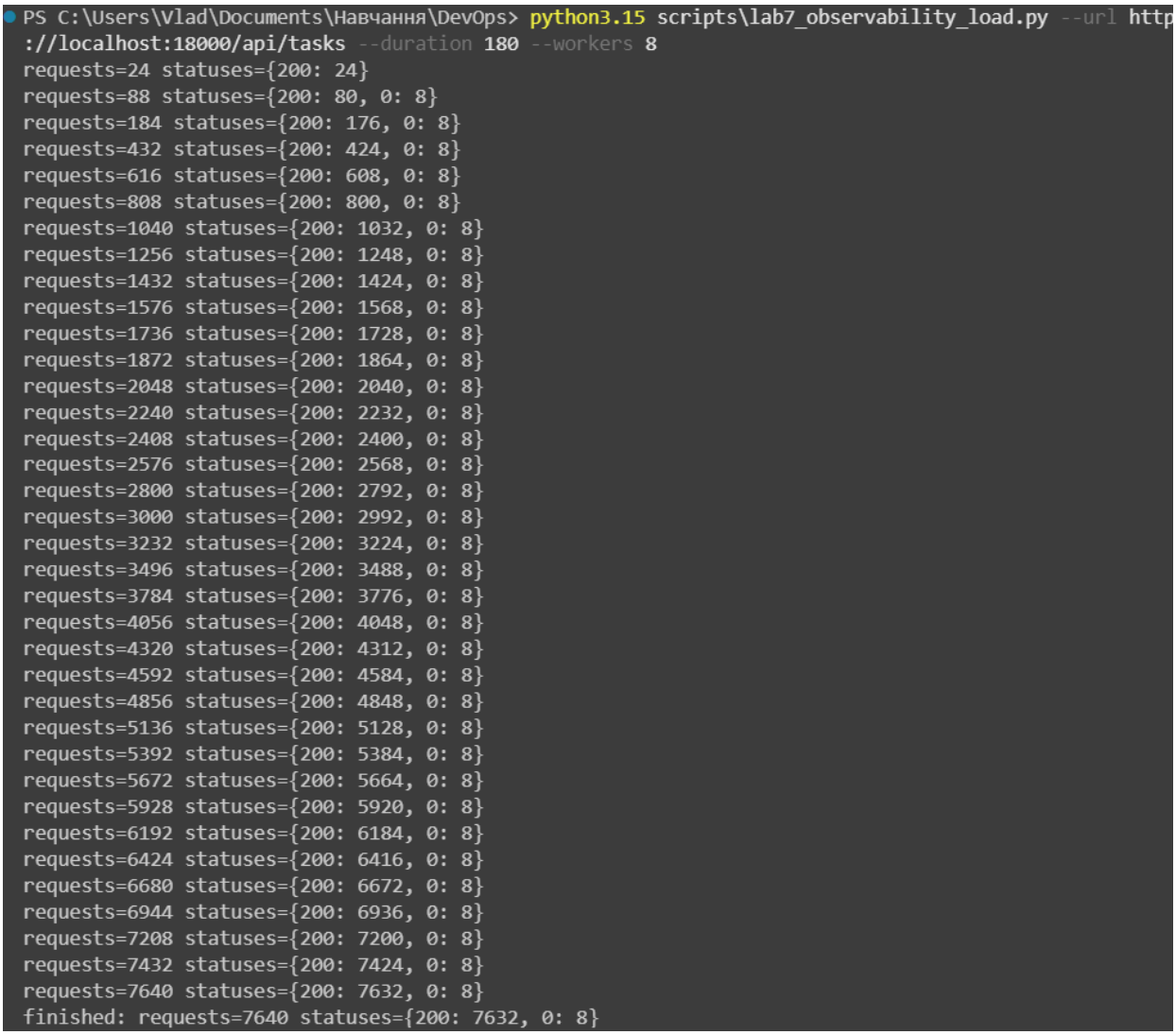


Рисунок 9. Результат генератора запитів

6 Припинення помилкового rollout

Завдання: перевірити canary-версію за порогом 95% успішних запитів і підтвердити автоматичний abort.

Виконання: AnalysisTemplate запитує Prometheus за міткою нового pod. Під навантаженням у Git тимчасово встановлено failRate: 0.8. Виміряна успішність canary 0,2053 нижча за поріг 0,95. AnalysisRun перейшов у Failed, Argo Rollouts зупинив ревізію. Значення failRate потім повернуто до 0 окремим Git-комітом.

Результат: помилкова ревізія ScaledDown; стабільна Healthy, готові 6/6 pod. Під час контрольного тесту було 786 HTTP 500 із 17 862 запитів.

```
PS C:\Users\Vlad\Documents\Навчання\DevOps> kubectl get analysisruns,rollouts -n trackhub
NAME                                                  STATUS   AGE
analysisrun.argoproj.io/trackhub-6dd65bc794-3-2       Failed   6d

NAME                              DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
rollout.argoproj.io/trackhub      6         6         6            6           11d
PS C:\Users\Vlad\Documents\Навчання\DevOps> kubectl argo rollouts get rollout trackhub -n trackhub
Name:            trackhub
Namespace:       trackhub
Status:          ✓ Healthy
Strategy:        Canary
  Step:          7/7
  SetWeight:     100
  ActualWeight:  100
Images:          ghcr.io/vlad8800/devops:1.3.1 (stable)
Replicas:
  Desired:       6
  Current:       6
  Updated:       6
  Ready:         6
  Available:     6

NAME                                KIND         STATUS        AGE   INFO
⟳ trackhub                          Rollout      ✓ Healthy     11d
├──# revision:4
│  └──⧉ trackhub-5d4cdfbd48         ReplicaSet   ✓ Healthy     6d1h  stable
│     ├──□ trackhub-5d4cdfbd48-7lzxb  Pod        ✓ Running     6d    ready:1/1,restarts:1
│     ├──□ trackhub-5d4cdfbd48-nzjhg  Pod        ✓ Running     6d    ready:1/1,restarts:1
│     ├──□ trackhub-5d4cdfbd48-s5kgs  Pod        ✓ Running     6d    ready:1/1,restarts:1
│     ├──□ trackhub-5d4cdfbd48-fqnmj  Pod        ✓ Running     6d    ready:1/1,restarts:1
│     ├──□ trackhub-5d4cdfbd48-ptjxl  Pod        ✓ Running     6d    ready:1/1,restarts:1
│     └──□ trackhub-5d4cdfbd48-k2nlw  Pod        ✓ Running     6d    ready:1/1,restarts:1
├──# revision:3
│  ├──⧉ trackhub-6dd65bc794         ReplicaSet   • ScaledDown  6d
│  └──α trackhub-6dd65bc794-3-2     AnalysisRun  ✕ Failed      6d    ✕ 1
└──# revision:1
   └──⧉ trackhub-84dc55f8d4         ReplicaSet   • ScaledDown  11d
PS C:\Users\Vlad\Documents\Навчання\DevOps>
```

Рисунок 10. Failed AnalysisRun і стабільний rollout

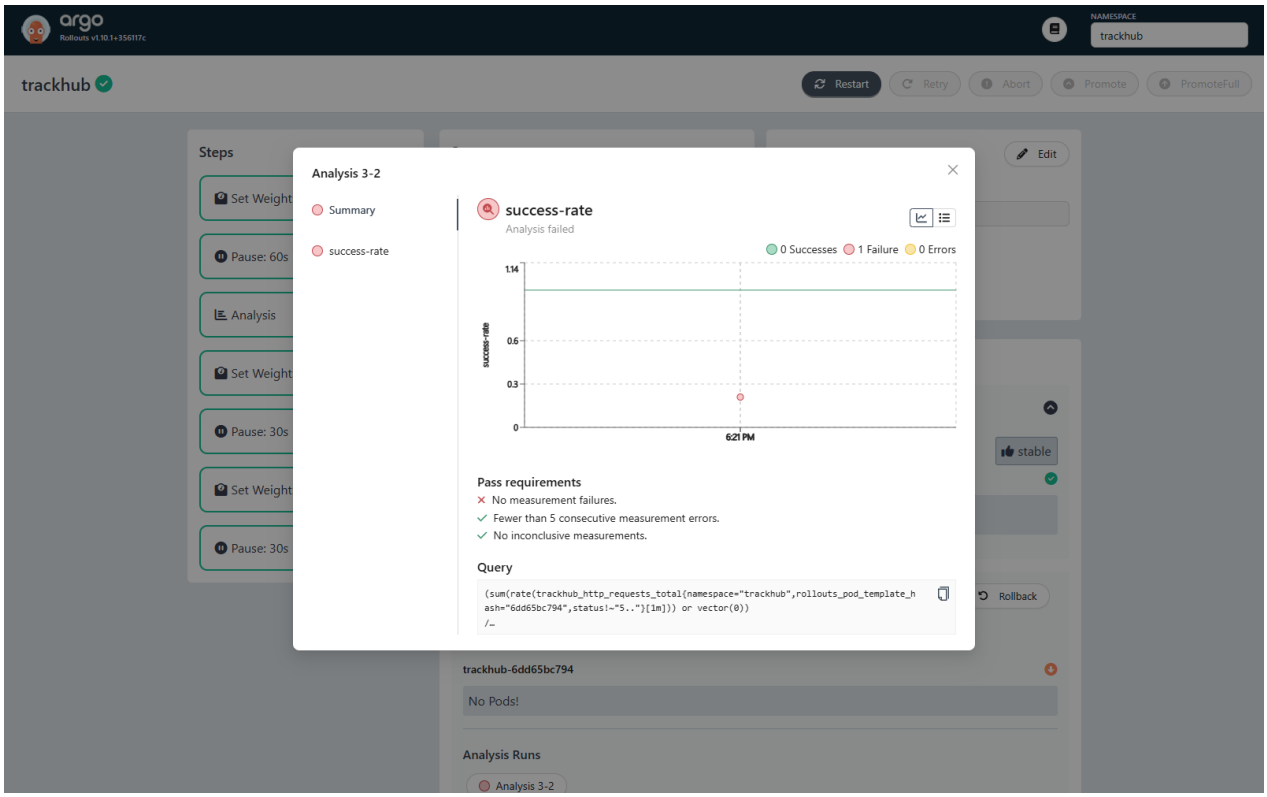


Рисунок 11. Вимірювання нижче порога успішності

7 Підсумкова демонстрація

Стан Argo CD показано на рисунку 1; збір метрик — на рисунках 3–4; логи — на рисунках 5–6; дашборд і навантаження — на рисунках 7–9; припинення помилкової версії та відновлення — на рисунках 10–11.

Автоматично виконано abort невдалої canary-ревізії. Повернення failRate у Git виконано окремим комітом. Після синхронізації Argo CD застосунок Healthy.

Код і машинні результати збережено в репозиторії: app/main.py, charts/trackhub, argocd-apps, scripts/lab7_observability_load.py, docs/observability-lab7/evidence.